Python Programming Fundamentals

Shop v2.0 — Lists of Objects

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1.1 A container for Product objects

```python
from product import Product

class Shop:
    def __init__(self, name: str):
        self.__name = name
        self.__products = []   # list of Product objects

    def get_name(self) -> str:
        return self.__name

    def get_all_products(self) -> list:
        return self.__products

    def __str__(self) -> str:
        return f"Shop: {self.__name} ({len(self.__products)} products)"
```

Outline

2. add_product()

```
class Shop:
    # ...
    def add_product(self, product: Product):
        if not isinstance(product, Product):
            raise TypeError("Must add a Product object")
        self.__products.append(product)
        return True
```

- `isinstance(product, Product)` checks the type
- `append()` adds to the end of the list
- Returns `True` so the caller knows it succeeded

Outline

3. get_all_products() and Iteration

```
class Shop:
    def get_all_products(self) -> list:
        return self.__products

    def display_all(self):
        if not self.__products:
            print("Shop is empty.")
            return
        print(f"\n--- {self.__name} ---")
        for product in self.__products:
            print(f"  {product}")
        print(f"  Total: {len(self.__products)} product(s)")
```

Iterating a list of objects is just like iterating a list of strings

3. get_all_products() and Iteration

`for product in self.__products:` gives you each Product in turn.

Outline

4.1 Searching the list

```
class Shop:
    def find_by_name(self, name: str):
        name = name.strip().lower()
        for product in self.__products:
            if product.get_name().lower() == name:
                return product
        return None   # not found
```

- `.lower()` makes the search case-insensitive
- Returns the **Product object** if found, `None` otherwise
- The caller checks: `if result is None:`

Outline

5. remove_product()

```
class Shop:
    def remove_product(self, name: str) -> bool:
        product = self.find_by_name(name)
        if product:
            self.__products.remove(product)
            return True
        return False
```

- Reuses `find_by_name()` — don't repeat yourself (DRY)
- `list.remove(item)` removes the **first** occurrence
- Returns `True` if removed, `False` if not found

Outline

6. total_value()

```
class Shop:
    def total_value(self) -> float:
        total = 0.0
        for product in self.__products:
            total += product.get_price()
        return total
```

Or using the built-in `sum()`:

```
    def total_value(self) -> float:
        return sum(p.get_price() for p in self.__products)
```

Both are correct — the `sum()` version is more “Pythonic”.

Outline

7. Complete Shop Class Summary

Method	Description
`add_product(p)`	Add a Product to the list
`get_all_products()`	Return the full list
`find_by_name(name)`	Search by name, return Product or None
`remove_product(name)`	Remove by name, return bool
`total_value()`	Sum of all product prices
`display_all()`	Print all products
`__str__()`	Summary string

Thanks for Watching - Any questions?